Chapter 2. Getting Started

The classic model of how free software projects get started was supplied by Eric Raymond, in a now-famous paper on open source processes entitled *The Cathedral and the Bazaar*. He wrote:

> *Every good work of software starts by scratching a developer's personal itch.*
>
> (from **http://www.catb.org/~esr/writings/cathedral-bazaar/**)

Note that Raymond wasn't saying that open source projects happen only when some individual gets an itch. Rather, he was saying that *good* software results when the programmer has a personal interest in seeing the problem solved; the relevance of this to free software was that a personal itch happened to be the most frequent motivation for starting a free software project.

This is still how most free software projects are started, but less so now than in 1997, when Raymond wrote those words. Today, we have the phenomenon of organizations—including for-profit corporations—starting large, centrally-managed open source projects from scratch. The lone programmer, banging out some code to solve a local problem and then realizing the result has wider applicability, is still the source of much new free software, but is not the only story.

Raymond's point is still insightful, however. The essential condition is that the producers of the software have a direct interest in its success, because they use it themselves. If the software doesn't do what it's supposed to do, the person or organization producing it will feel the dissatisfaction in their daily work. For example, the OpenAdapter project (http://www.openadapter.org/), which was started by investment bank Dresdner Kleinwort Wasserstein as an open source framework for integrating disparate financial information systems, can hardly be said to scratch any individual programmer's personal itch. It scratches an institutional itch. But that itch arises directly from the experiences of the institution and its partners, and therefore if the project fails to relieve them, they will know. This arrangement produces good software because the feedback loop flows in the right direction. The program isn't being written to be sold to someone else so they can solve *their* problem. It's being written to solve one's *own* problem, and then shared with everyone, much as though the problem were a disease and the software were medicine whose distribution is meant to completely eradicate the epidemic.

This chapter is about how to introduce a new free software project to the world, but many of its recommendations would sound familiar to a health organization distributing medicine. The goals are very similar: you want to make it clear what the medicine does, get it into the hands of the right people, and make sure that those who receive it know how to use it. But with software, you also want to entice some of the recipients into joining the ongoing research effort to improve the medicine.

Free software distribution is a twofold task. The software needs to acquire users, and to acquire developers. These two needs are not necessarily in conflict, but they do add some complexity to a project's initial presentation. Some information is useful for both audiences, some is useful only for one or the other. Both kinds of information should subscribe to the principle of scaled presentation; that is, the degree of detail presented at each stage should correspond directly to the amount of time and effort put in by the reader. More effort should always equal more reward. When the two do not correlate tightly, people may quickly lose faith and stop investing effort.

The corollary to this is that *appearances matter*. Programmers, in particular, often don't like to believe this. Their love of substance over form is almost a point of professional pride. It's no accident that so many programmers exhibit an antipathy for marketing and public relations work, nor that professional graphic designers are often horrified at what programmers come up with on their own.

This is a pity, because there are situations where form *is* substance, and project presentation is one of them. For example, the very first thing a visitor learns about a project is what its web site looks like. This information is absorbed before any of the actual content on the site is comprehended—before any of the text has been read or links clicked on. However unjust it may be, people cannot stop themselves

from forming an immediate first impression. The site's appearance signals whether care was taken in organizing the project's presentation. Humans have extremely sensitive antennae for detecting the investment of care. Most of us can tell in one glance whether a web site was thrown together quickly or was given serious thought. This is the first piece of information your project puts out, and the impression it creates will carry over to the rest of the project by association.

Thus, while much of this chapter talks about the content your project should start out with, remember that its look and feel matter too. Because the project web site has to work for two different types of visitors—users and developers—special attention must be paid to clarity and directedness. Although this is not the place for a general treatise on web design, one principle is important enough to deserve mention, particularly when the site serves multiple (if overlapping) audiences: people should have a rough idea where a link goes before clicking on it. For example, it should be obvious *from looking at the links* to user documentation that they lead to user documentation, and not to, say, developer documentation. Running a project is partly about supplying information, but it's also about supplying comfort. The mere presence of certain standard offerings, in expected places, reassures users and developers who are deciding whether they want to get involved. It says that this project has its act together, has anticipated the questions people will ask, and has made an effort to answer them in a way that requires minimal exertion on the part of the asker. By giving off this aura of preparedness, the project sends out a message: "Your time will not be wasted if you get involved," which is exactly what people need to hear.

But First, Look Around

Before starting an open source project, there is one important caveat:

Always look around to see if there's an existing project that does what you want. The chances are pretty good that whatever problem you want solved now, someone else wanted solved before you. If they did solve it, and released their code under a free license, then there's no reason for you to reinvent the wheel today. There are exceptions, of course: if you want to start a project as an educational experience, preexisting code won't help; or maybe the project you have in mind is so specialized that you know there is zero chance anyone else has done it. But generally, there's no point not looking, and the payoff can be huge. If the usual Internet search engines don't turn up anything, try searching on http://freshmeat.net/ (an open source project news site, about which more will be said later), on http://www.sourceforge.net/, and in the Free Software Foundation's directory of free software at http://directory.fsf.org/.

Even if you don't find exactly what you were looking for, you might find something so close that it makes more sense to join that project and add functionality than to start from scratch yourself.

Starting From What You Have

You've looked around, found that nothing out there really fits your needs, and decided to start a new project.

What now?

The hardest part about launching a free software project is transforming a private vision into a public one. You or your organization may know perfectly well what you want, but expressing that goal comprehensibly to the world is a fair amount of work. It is essential, however, that you take the time to do it. You and the other founders must decide what the project is really about—that is, decide its limitations, what it *won't* do as well as what it will—and write up a mission statement. This part is usually not too hard, though it can sometimes reveal unspoken assumptions and even disagreements about the nature of the project, which is fine: better to resolve those now than later. The next step is to package up the project for public consumption, and this is, basically, pure drudgery.

What makes it so laborious is that it consists mainly of organizing and documenting things everyone already knows—"everyone", that is, who's been involved in the project so far. Thus, for the people doing the work, there is no immediate benefit. They do not need a `README` file giving an overview of the project, nor a design document or user manual. They do not need a carefully arranged code tree con-

forming to the informal but widespread standards of software source distributions. Whatever way the source code is arranged is fine for them, because they're already accustomed to it anyway, and if the code runs at all, they know how to use it. It doesn't even matter, for them, if the fundamental architectural assumptions of the project remain undocumented; they're already familiar with that too.

Newcomers, on the other hand, need these things. Fortunately, they don't need them all at once. It's not necessary for you to provide every possible resource before taking a project public. In a perfect world, perhaps, every new open source project would start out life with a thorough design document, a complete user manual (with special markings for features planned but not yet implemented), beautifully and portably packaged code, capable of running on any computing platform, and so on. In reality, taking care of all these loose ends would be prohibitively time-consuming, and anyway, it's work that one can reasonably hope volunteers will help with once the project is under way.

What *is* necessary, however, is that enough investment be put into presentation that newcomers can get past the initial obstacle of unfamiliarity. Think of it as the first step in a bootstrapping process, to bring the project to a kind of minimum activation energy. I've heard this threshold called the *hacktivation energy*: the amount of energy a newcomer must put in before she starts getting something back. The lower a project's hacktivation energy, the better. Your first task is bring the hacktivation energy down to a level that encourages people to get involved.

Each of the following subsections describes one important aspect of starting a new project. They are presented roughly in the order that a new visitor would encounter them, though of course the order in which you actually implement them might be different. You can treat them as a checklist. When starting a project, just go down the list and make sure you've got each item covered, or at least that you're comfortable with the potential consequences if you've left one out.

Choose a Good Name

Put yourself in the shoes of someone who's just heard about your project, perhaps by having stumbled across it while searching for software to solve some problem. The first thing they'll encounter is the project's name.

A good name will not automatically make your project successful, and a bad name will not doom it—well, a *really* bad name probably could do that, but we start from the assumption that no one here is actively trying to make their project fail. However, a bad name can slow down adoption of the project, either because people don't take it seriously, or because they simply have trouble remembering it.

A good name:

- Gives some idea what the project does, or at least is related in an obvious way, such that if one knows the name and knows what the project does, the name will come quickly to mind thereafter.
- Is easy to remember. Here, there is no getting around the fact that English has become the default language of the Internet: "easy to remember" means "easy for someone who can read English to remember." Names that are puns dependent on native-speaker pronounciation, for example, will be opaque to the many non-native English readers out there. If the pun is particularly compelling and memorable, it may still be worth it; just keep in mind that many people seeing the name will not hear it in their head the way a native speaker would.
- Is not the same as some other project's name, and does not infringe on any trademarks. This is just good manners, as well as good legal sense. You don't want to create identity confusion. It's hard enough to keep track of everything that's available on the Net already, without different things having the same name.

  The resources mentioned earlier in the section called “But First, Look Around” are useful in discovering whether another project already has the name you're thinking of. Free trademark searches are available at http://www.nameprotect.org/ and http://www.uspto.gov/.

- If possible, is available as a domain name in the `.com`, `.net`, and `.org` top-level domains. You should pick one, probably `.org`, to advertise as the official home site for the project; the other two should forward there and are simply to prevent third parties from creating identity confusion around the project's name. Even if you intend to host the project at some other site (see the section called “Canned Hosting”), you can still register project-specific domains and forward them to the hosting site. It helps users a lot to have a simple URL to remember.

Have a Clear Mission Statement

Once they've found the project's web site, the next thing people will look for is a quick description, a mission statement, so they can decide (within 30 seconds) whether or not they're interested in learning more. This should be prominently placed on the front page, preferably right under the project's name.

The mission statement should be concrete, limiting, and above all, short. Here's an example of a good one, from http://www.openoffice.org/:

> *To create, as a community, the leading international office suite that will run on all major platforms and provide access to all functionality and data through open-component based APIs and an XML-based file format.*

In just a few words, they've hit all the high points, largely by drawing on the reader's prior knowledge. By saying "*as a community*", they signal that no one corporation will dominate development; "*international*" means that the software will allow people to work in multiple languages and locales; "*all major platforms*" means it will be portable to Unix, Macintosh, and Windows. The rest signals that open interfaces and easily understandable file formats are an important part of the goal. They don't come right out and say that they're trying to be a free alternative to Microsoft Office, but most people can probably read between the lines. Although this mission statement looks broad at first glance, in fact it is quite circumscribed: the words "*office suite*" mean something very concrete to those familiar with such software. Again, the reader's presumed prior knowledge (in this case probably from MS Office) is used to keep the mission statement concise.

The nature of a mission statement depends partly on who is writing it, not just on the software it describes. For example, it makes sense for OpenOffice.org to use the words "*as a community*", because the project was started, and is still largely sponsored, by Sun Microsystems. By including those words, Sun indicates its sensitivity to worries that it might try to dominate the development process. With this sort of thing, merely demonstrating awareness of the *potential* for a problem goes a long way toward avoiding the problem entirely. On the other hand, projects that aren't sponsored by a single corporation probably don't need such language; after all, development by community is the norm, so there would ordinarily be no reason to list it as part of the mission.

State That the Project is Free

Those who remain interested after reading the mission statement will next want to see more details, perhaps some user or developer documentation, and eventually will want to download something. But before any of that, they'll need to be sure it's open source.

The front page must make it unambiguously clear that the project is open source. This may seem obvious, but you would be surprised how many projects forget to do it. I have seen free software project web sites where the front page not only did not say which particular free license the software was distributed under, but did not even state outright that the software was free at all. Sometimes the crucial bit of information was relegated to the Downloads page, or the Developers page, or some other place that required one more mouse click to get to. In extreme cases, the license was not given anywhere on the web site at all—the only way to find it was to download the software and look inside.

Don't make this mistake. Such an omission can lose many potential developers and users. State up front,

right below the mission statement, that the project is "free software" or "open source software", and give the exact license. A quick guide to choosing a license is given in the section called “Choosing a License and Applying It” later in this chapter, and licensing issues are discussed in detail in Chapter 9, *Licenses, Copyrights, and Patents*.

At this point, our hypothetical visitor has determined—probably in a minute or less—that she's interested in spending, say, at least five more minutes investigating this project. The next sections describe what she should encounter in that five minutes.

Features and Requirements List

There should be a brief list of the features the software supports (if something isn't completed yet, you can still list it, but put "*planned*" or "*in progress*" next to it), and the kind of computing environment required to run the software. Think of the features/requirements list as what you would give to someone asking for a quick summary of the software. It is often just a logical expansion of the mission statement. For example, the mission statement might say:

> *To create a full-text indexer and search engine with a rich API, for use by programmers in providing search services for large collections of text files.*

The features and requirements list would give the details, clarifying the mission statement's scope:

> *Features:*
>
> - *Searches plain text, HTML, and XML*
> - *Word or phrase searching*
> - *(planned) Fuzzy matching*
> - *(planned) Incremental updating of indexes*
> - *(planned) Indexing of remote web sites*
>
> *Requirements:*
>
> - *Python 2.2 or higher*
> - *Enough disk space to hold the indexes (approximately 2x original data size)*

With this information, readers can quickly get a feel for whether this software has any hope of working for them, and they can consider getting involved as developers too.

Development Status

People always want to know how a project is doing. For new projects, they want to know the gap between the project's promise and current reality. For mature projects, they want to know how actively it is maintained, how often it puts out new releases, how responsive it is likely to be to bug reports, etc.

To answer these questions, you should provide a development status page, listing the project's near-term goals and needs (for example, it might be looking for developers with a particular kind of expertise). The page can also give a history of past releases, with feature lists, so visitors can get an idea of how the

project defines "progress" and how quickly it makes progress according to that definition.

Don't be afraid of looking unready, and don't give in to the temptation to hype the development status. Everyone knows that software evolves by stages; there's no shame in saying "This is alpha software with known bugs. It runs, and works at least some of the time, but use at your own risk." Such language won't scare away the kinds of developers you need at that stage. As for users, one of the worst things a project can do is attract users before the software is ready for them. A reputation for instability or bugginess is very hard to shake, once acquired. Conservativism pays off in the long run; it's always better for the software to be *more* stable than the user expected than less, and pleasant surprises produce the best kind of word-of-mouth.

Alpha and Beta

The term *alpha* usually means a first release, with which users can get real work done and which has all the intended functionality, but which also has known bugs. The main purpose of alpha software is to generate feedback, so the developers know what to work on. The next stage, *beta*, means the software has had all the serious bugs fixed, but has not yet been tested enough to certify for release. The purpose of beta software is to either become the official release, assuming no bugs are found, or provide detailed feedback to the developers so they can reach the official release quickly. The difference between alpha and beta is very much a matter of judgement.

Downloads

The software should be downloadable as source code in standard formats. When a project is first getting started, binary (executable) packages are not necessary, unless the software has such complicated build requirements or dependencies that merely getting it to run would be a lot of work for most people. (But if this is the case, the project is going to have a hard time attracting developers anyway!)

The distribution mechanism should be as convenient, standard, and low-overhead as possible. If you were trying to eradicate a disease, you wouldn't distribute the medicine in such a way that it requires a non-standard syringe size to administer. Likewise, software should conform to standard build and installation methods; the more it deviates from the standards, the more potential users and developers will give up and go away confused.

That sounds obvious, but many projects don't bother to standardize their installation procedures until very late in the game, telling themselves they can do it any time: *"We'll sort all that stuff out when the code is closer to being ready."* What they don't realize is that by putting off the boring work of finishing the build and installation procedures, they are actually making the code take longer to get ready—because they discourage developers who might otherwise have contributed to the code. Most insidiously, they don't *know* they're losing all those developers, because the process is an accumulation of non-events: someone visits a web site, downloads the software, tries to build it, fails, gives up and goes away. Who will ever know it happened, except the person themselves? No one working on the project will realize that someone's interest and good will have been silently squandered.

Boring work with a high payoff should always be done early, and significantly lowering the project's barrier to entry through good packaging brings a very high payoff.

When you release a downloadable package, it is vital that you give a unique version number to the release, so that people can compare any two releases and know which supersedes the other. A detailed discussion of version numbering can be found in the section called “Release Numbering”, and the details of standardizing build and installation procedures are covered in the section called “Packaging”, both in Chapter 7, *Packaging, Releasing, and Daily Development*.

Version Control and Bug Tracker Access

Downloading source packages is fine for those who just want to install and use the software, but it's not enough for those who want to debug or add new features. Nightly source snapshots can help, but they're still not fine-grained enough for a thriving development community. People need real-time access to the latest sources, and the way to give them that is to use a version control system. The presence of anonymously accessible version controlled sources is a sign—to both users and developers—that this project is making an effort to give people what they need to participate. If you can't offer version control right away, then put up a sign saying you intend to set it up soon. Version control infrastructure is discussed in detail in the section called “Version Control” in Chapter 3, *Technical Infrastructure*.

The same goes for the project's bug tracker. The importance of a bug tracking system lies not only in its usefulness to developers, but in what it signifies for project observers. For many people, an accessible bug database is one of the strongest signs that a project should be taken seriously. Furthermore, the higher the number of bugs in the database, the better the project looks. This might seem counterintuitive, but remember that the number of bugs recorded really depends on three things: the absolute number of bugs present in the software, the number of users using the software, and the convenience with which those users can register new bugs. Of these three factors, the latter two are more significant than the first. Any software of sufficient size and complexity has an essentially arbitrary number of bugs waiting to be discovered. The real question is, how well will the project do at recording and prioritizing those bugs? A project with a large and well-maintained bug database (meaning bugs are responded to promptly, duplicate bugs are unified, etc.) therefore makes a better impression than a project with no bug database, or a nearly empty database.

Of course, if your project is just getting started, then the bug database will contain very few bugs, and there's not much you can do about that. But if the status page emphasizes the project's youth, and if people looking at the bug database can see that most filings have taken place recently, they can extrapolate from that that the project still has a healthy *rate* of filings, and they will not be unduly alarmed by the low absolute number of bugs recorded.

Note that bug trackers are often used to track not only software bugs, but enhancement requests, documentation changes, pending tasks, and more. The details of running a bug tracker are covered in the section called “Bug Tracker” in Chapter 3, *Technical Infrastructure*, so I won't go into them here. The important thing from a presentation point of view is just to *have* a bug tracker, and to make sure that fact is visible from the front page of the project.

Communications Channels

Visitors usually want to know how to reach the human beings involved with the project. Provide the addresses of mailing lists, chat rooms, and IRC channels, and any other forums where others involved with the software can be reached. Make it clear that you and the other authors of the project are subscribed to these mailing lists, so people see there's a way to give feedback that will reach the developers. Your presence on the lists does not imply a committment to answer all questions or implement all feature requests. In the long run, most users will probably never join the forums anyway, but they will be comforted to know that they *could* if they ever needed to.

In the early stages of a project, there's no need to have separate user and developer forums. It's much better to have everyone involved with the software talking together, in one "room." Among early adopters, the distinction between developer and user is often fuzzy; to the extent that the distinction can be made, the ratio of developers to users is usually much higher in the early days of the project than later on. While you can't assume that every early adopter is a programmer who wants to hack on the software, you can assume that they are at least interested in following development discussions and in getting a sense of the project's direction.

As this chapter is only about getting a project started, it's enough merely to say that these communications forums need to exist. Later, in the section called “Handling Growth” in Chapter 6, *Communications*, we'll examine where and how to set up such forums, the ways in which they might need moderation or other management, and how to separate user forums from developer forums, when the time comes, without creating an unbridgeable gulf.

Developer Guidelines

If someone is considering contributing to the project, she'll look for developer guidelines. Developer guidelines are not so much technical as social: they explain how the developers interact with each other and with the users, and ultimately how things get done.

This topic is covered in detail in the section called “Writing It All Down” in Chapter 4, *Social and Political Infrastructure*, but the basic elements of developer guidelines are:

- pointers to forums for interaction with other developers
- instructions on how to report bugs and submit patches
- some indication of *how* development is usually done—is the project a benevolent dictatorship, a democracy, or something else

No pejorative sense is intended by "dictatorship", by the way. It's perfectly okay to run a tyranny where one particular developer has veto power over all changes. Many successful projects work this way. The important thing is that the project come right out and say so. A tyranny pretending to be a democracy will turn people off; a tyranny that says it's a tyranny will do fine as long as the tyrant is competent and trusted.

See http://svn.collab.net/repos/svn/trunk/www/hacking.html for an example of particularly thorough developer guidelines, or http://www.openoffice.org/dev_docs/guidelines.html for broader guidelines that focus more on governance and the spirit of participation and less on technical matters.

The separate issue of providing a programmer's introduction to the software is discussed in the section called “Developer documentation” later in this chapter.

Documentation

Documentation is essential. There needs to be *something* for people to read, even if it's rudimentary and incomplete. This falls squarely into the "drudgery" category referred to earlier, and is often the first area where a new open source project falls down. Coming up with a mission statement and feature list, choosing a license, summarizing development status—these are all relatively small tasks, which can be definitively completed and usually need not be returned to once done. Documentation, on the other hand, is never really finished, which may be one reason people sometimes delay starting it at all.

The most insidious thing is that documentation's utility to those writing it is the reverse of its utility to those who will read it. The most important documentation for initial users is the basics: how to quickly set up the software, an overview of how it works, perhaps some guides to doing common tasks. Yet these are exactly the things the *writers* of the documentation know all too well—so well that it can be difficult for them to see things from the reader's point of view, and to laboriously spell out the steps that (to the writers) seem so obvious as to be unworthy of mention.

There's no magic solution to this problem. Someone just needs to sit down and write the stuff, and then run it by typical new users to test its quality. Use a simple, easy-to-edit format such as HTML, plain text, Texinfo, or some variant of XML—something that's convenient for lightweight, quick improvements on the spur of the moment. This is not only to remove any overhead that might impede the original writers from making incremental improvements, but also for those who join the project later and want to work on the documentation.

One way to ensure basic initial documentation gets done is to limit its scope in advance. That way, writing it at least won't feel like an open-ended task. A good rule of thumb is that it should meet the following minimal criteria:

- Tell the reader clearly how much technical expertise they're expected to have.
- Describe clearly and thoroughly how to set up the software, and somewhere near the beginning of the documentation, tell the user how to run some sort of diagnostic test or simple command to confirm that they've set things up correctly. Startup documentation is in some ways more important than actual usage documentation. The more effort someone has invested in installing and getting started with the software, the more persistent she'll be in figuring out advanced functionality that's not well-documented. When people abandon, they abandon early; therefore, it's the earliest stages, like installation, that need the most support.
- Give one tutorial-style example of how to do a common task. Obviously, many examples for many tasks would be even better, but if time is limited, pick one task and walk through it thoroughly. Once someone sees that the software *can* be used for one thing, they'll start to explore what else it can do on their own—and, if you're lucky, start filling in the documentation themselves. Which brings us to the next point...
- Label the areas where the documentation is known to be incomplete. By showing the readers that you are aware of its deficiencies, you align yourself with their point of view. Your empathy reassures them that they don't face a struggle to convince the project of what's important. These labels needn't represent promises to fill in the gaps by any particular date —it's equally legitimate to treat them as open requests for volunteer help.

The last point is of wider importance, actually, and can be applied to the entire project, not just the documentation. An accurate accounting of known deficiencies is the norm in the open source world. You don't have to exaggerate the project's shortcomings, just identify them scrupulously and dispassionately when the context calls for it (whether in the documentation, in the bug tracking database, or on a mailing list discussion). No one will treat this as defeatism on the part of the project, nor as a commitment to solve the problems by a certain date, unless the project makes such a commitment explicitly. Since anyone who uses the software will discover the deficiencies for themselves, it's much better for them to be psychologically prepared—then the project will look like it has a solid knowledge of how it's doing.

Maintaining a FAQ

A *FAQ* ("Frequently Asked Questions" document) can be one of the best investments a project makes in terms of educational payoff. FAQs are highly tuned to the questions users and developers actually ask—as opposed to the questions you might have *expected* them to ask—and therefore, a well-maintained FAQ tends to give those who consult it exactly what they're looking for. The FAQ is often the first place users look when they encounter a problem, often even in preference to the official manual, and it's probably the document in your project most likely to be linked to from other sites.

Unfortunately, you cannot make the FAQ at the start of the project. Good FAQs are not written, they are grown. They are by definition reactive documents, evolving over time in response to people's day-to-day usage of the software. Since it's impossible to correctly anticipate the questions people will ask, it is impossible to sit down and write a useful FAQ from scratch.

Therefore, don't waste your time trying to. You may, however, find it useful to set up a mostly blank FAQ template, so there will be an obvious place for people to contribute questions and answers after the project is under way. At this stage, the most important property is not completeness, but convenience: if the FAQ is easy to add to, people will add to it. (Proper FAQ maintenance is a non-trivial and intriguing problem, and is discussed more in the section called “FAQ Manager” in Chapter 8, *Managing Volunteers*.)

Availability of documentation

Documentation should be available from two places: online (directly from the web site), *and* in the downloadable distribution of the software (see the section called “Packaging” in Chapter 7, *Packaging, Releasing, and Daily Development*). It needs to be online, in browsable form, because people often read documentation *before* downloading software for the first time, as a way of helping them decide whether to download at all. But it should also accompany the software, on the principle that downloading should supply (i.e., make locally accessible) everything one needs to use the package.

For online documentation, make sure that there is a link that brings up the *entire* documentation in one HTML page (put a note like "monolithic" or "all-in-one" or "single large page" next to the link, so people know that it might take a while to load). This is useful because people often want to search for a specific word or phrase across the entire documentation. Generally, they already know what they're looking for; they just can't remember what section it's in. For such people, nothing is more frustrating than encountering one HTML page for the table of contents, then a different page for the introduction, then a different page for installation instructions, etc. When the pages are broken up like that, their browser's search function is useless. The separate-page style is useful for those who already know what section they need, or who want to read the entire documentation from front to back in sequence. But this is *not* the most common way documentation is accessed. Far more often, someone who is basically familiar with the software is coming back to search for a specific word or phrase. To fail to provide them with a single, searchable document would only make their lives harder.

Developer documentation

Developer documentation is written to help programmers understand the code, so they can repair and extend it. This is somewhat different from the *developer guidelines* discussed earlier, which are more social than technical. Developer guidelines tell programmers how to get along with each other; developer documentation tells them how to get along with the code itself. The two are often packaged together in one document for convenience (as with the http://svn.collab.net/repos/svn/trunk/www/hacking.html example given earlier), but they don't have to be.

Although developer documentation can be very helpful, there's no reason to delay a release to do it. As long as the original authors are available (and willing) to answer questions about the code, that's enough to start with. In fact, having to answer the same questions over and over is a common motivation for writing documentation. But even before it's written, determined contributors will still manage to find their way around the code. The force that drives people to spend time learning a code base is that the code does something useful for them. If people have faith in that, they will take the time to figure things out; if they don't have that faith, no amount of developer documentation will get or keep them.

So if you have time to write documentation for only one audience, write it for users. All user documentation is, in effect, developer documentation as well; any programmer who's going to work on a piece of software will need to be familiar with how to use it. Later, when you see programmers asking the same questions over and over, take the time to write up some separate documents just for them.

Some projects use wikis for their initial documentation, or even as their primary documentation. In my experience, this really only works if the wiki is actively edited by a few people who agree on how the documentation is to be organized and what sort of "voice" it should have. See the section called “Wikis” in Chapter 3, *Technical Infrastructure* for more.

Example Output and Screenshots

If the project involves a graphical user interface, or if it produces graphical or otherwise distinctive output, put some samples up on the project web site. In the case of interface, this means screenshots; for output, it might be screenshots or just files. Both cater to people's need for instant gratification: a single screenshot can be more convincing than paragraphs of descriptive text and mailing list chatter, because a screenshot is inarguable proof that the software *works*. It may be buggy, it may be hard to install, it may be incompletely documented, but that screenshot is still proof that if one puts in enough effort, one can get it to run.

Screenshots

Since screenshots can be daunting until you've actually made a few, here are basic instructions for making them. Using the Gimp (http://www.gimp.org/), open File->Acquire->Screenshot, choose Single Window or Whole Screen, then click OK. Now your next mouse click will capture the window or screen clicked on as an image in the Gimp. Crop and resize the image as necessary, using the instructions at http://www.gimp.org/tutorials/Lite_Quickies/#crop.

There are many other things you could put on the project web site, if you have the time, or if for one reason or another they are especially appropriate: a news page, a project history page, a related links page, a site-search feature, a donations link, etc. None of these are necessities at startup time, but keep them in mind for the future.

Canned Hosting

There are a few sites that provide free hosting and infrastructure for open source projects: a web area, version control, a bug tracker, a download area, chat forums, regular backups, etc. The details vary from site to site, but the same basic services are offered at all of them. By using one of these sites, you get a lot for free; what you give up, obviously, is fine-grained control over the user experience. The hosting service decides what software the site runs, and may control or at least influence the look and feel of the project's web pages.

See the section called “Canned Hosting” in Chapter 3, *Technical Infrastructure* for a more detailed discussion of the advantages and disadvantages of canned hosting, and a list of sites that offer it.

Choosing a License and Applying It

This section is intended to be a very quick, very rough guide to choosing a license. Read Chapter 9, *Licenses, Copyrights, and Patents* to understand the detailed legal implications of the different licenses, and how the license you choose can affect people's ability to mix your software with other free software.

There are a great many free software licenses to choose from. Most of them we needn't consider here, as they were written to satisfy the particular legal needs of some corporation or person, and wouldn't be appropriate for your project. We will restrict ourselves to just the most commonly used licenses; in most cases, you will want to choose one of them.

The "Do Anything" Licenses

If you're comfortable with your project's code potentially being used in proprietary programs, then use an *MIT/X-style* license. It is the simplest of several minimal licenses that do little more than assert nominal copyright (without actually restricting copying) and specify that the code comes with no warranty. See the section called “The MIT / X Window System License” for details.

The GPL

If you don't want your code to be used in proprietary programs, use the GNU General Public License (http://www.gnu.org/licenses/gpl.html). The GPL is probably the most widely recognized free software license in the world today. This is in itself a big advantage, since many potential users and contributors will already be familiar with it, and therefore won't have to spend extra time to read and understand your license. See the section called “The GNU General Public License” in Chapter 9, *Licenses, Copyrights, and Patents* for details.

How to Apply a License to Your Software

Once you've chosen a license, you should state it on the project's front page. You don't need to include the actual text of the license there; just give the name of the license, and make it link to the full license text on another page.

This tells the public what license you *intend* the software to be released under, but it's not sufficient for legal purposes. For that, the software itself must contain the license. The standard way to do this is to put the full license text in a file called `COPYING` (or `LICENSE`), and then put a short notice at the top of each source file, naming the copyright date, holder, and license, and saying where to find the full text of the license.

There are many variations on this pattern, so we'll look at just one example here. The GNU GPL says to put a notice like this at the top of each source file:

```
Copyright (C) <year>  <name of author>

This program is free software; you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation; either version 2 of the License, or
(at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with this program; if not, write to the Free Software
Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA  02111-1307  USA
```

It does not say specifically that the copy of the license you received along with the program is in the file `COPYING`, but that's where it's usually put. (You could change the above notice to state that directly.) This template also gives a geographical address from which to request a copy of the license. Another common method is to give a link to a web page containing the license. Just use your judgement and point to wherever you feel the most permanent copy of the license is maintained, which might simply be somewhere on your project's web site. In general, the notice you put in each source file does not have to look exactly like the one above, as long as it starts with the same notice of copyright holder and date, states the name of the license, and makes it clear where to view the full license.

Setting the Tone

So far we've covered one-time tasks you do during project setup: picking a license, arranging the initial web site, etc. But the most important aspects of starting a new project are dynamic. Choosing a mailing list address is easy; ensuring that the list's conversations remain on-topic and productive is another matter entirely. If the project is being opened up after years of closed, in-house development, its development processes will change, and you will have to prepare the existing developers for that change.

The first steps are the hardest, because precedents and expectations for future conduct have not yet been set. Stability in a project does not come from formal policies, but from a shared, hard-to-pin-down collective wisdom that develops over time. There are often written rules as well, but they tend to be essentially a distillation of the intangible, ever-evolving agreements that really guide the project. The written policies do not define the project's culture so much as describe it, and even then only approximately.

There are a few reasons why things work out this way. Growth and high turnover are not as damaging to the accumulation of social norms as one might think. As long as change does not happen *too* quickly, there is time for new arrivals to learn how things are done, and after they learn, they will help reinforce

those ways themselves. Consider how children's songs survive the centuries. There are children today singing roughly the same rhymes as children did hundreds of years ago, even though there are no children alive now who were alive then. Younger children hear the songs sung by older ones, and when they are older, they in turn will sing them in front of other younger ones. The children are not engaging in a conscious program of transmission, of course, but the reason the songs survive is nonetheless that they are transmitted regularly and repeatedly. The time scale of free software projects may not be measured in centuries (we don't know yet), but the dynamics of transmission are much the same. The turnover rate is faster, however, and must be compensated for by a more active and deliberate transmission effort.

This effort is aided by the fact that people generally show up expecting and looking for social norms. That's just how humans are built. In any group unified by a common endeavor, people who join instinctively search for behaviors that will mark them as part of the group. The goal of setting precedents early is to make those "in-group" behaviors be ones that are useful to the project; for once established, they will be largely self-perpetuating.

Following are some examples of specific things you can do to set good precedents. They're not meant as an exhaustive list, just as illustrations of the idea that setting a collaborative mood early helps a project tremendously. Physically, every developer may be working alone in a room by themselves, but you can do a lot to make them *feel* like they're all working together in the same room. The more they feel this way, the more time they'll want to spend on the project. I chose these particular examples because they came up in the Subversion project (http://subversion.tigris.org/), which I participated in and observed from its very beginning. But they're not unique to Subversion; situations like these will come up in most open source projects, and should be seen as opportunities to start things off on the right foot.

Avoid Private Discussions

Even after you've taken the project public, you and the other founders will often find yourselves wanting to settle difficult questions by private communications among an inner circle. This is especially true in the early days of the project, when there are so many important decisions to make, and, usually, few volunteers qualified to make them. All the obvious disadvantages of public list discussions will loom palpably in front of you: the delay inherent in email conversations, the need to leave sufficient time for consensus to form, the hassle of dealing with naive volunteers who think they understand all the issues but actually don't (every project has these; sometimes they're next year's star contributors, sometimes they stay naive forever), the person who can't understand why you only want to solve problem X when it's obviously a subset of larger problem Y, and so on. The temptation to make decisions behind closed doors and present them as *faits accomplis*, or at least as the firm recommendations of a united and influential voting block, will be great indeed.

Don't do it.

As slow and cumbersome as public discussions can be, they're almost always preferable in the long run. Making important decisions in private is like spraying contributor repellant on your project. No serious volunteer would stick around for long in an environment where a secret council makes all the big decisions. Furthermore, public discussion has beneficial side effects that will last beyond whatever ephemeral technical question was at issue:

- The discussion will help train and educate new developers. You never know how many eyes are watching the conversation; even if most people don't participate, many may be tracking silently, gleaning information about the software.
- The discussion will train *you* in the art of explaining technical issues to people who are not as familiar with the software as you are. This is a skill that requires practice, and you can't get that practice by talking to people who already know what you know.
- The discussion and its conclusions will be available in public archives forever after, enabling future discussions to avoid retracing the same steps. See the section called “Conspicuous Use of Archives” in Chapter 6, *Communications*.

Finally, there is the possibility that someone on the list may make a real contribution to the conversation, by coming up with an idea you never anticipated. It's hard to say how likely this is; it just depends on the complexity of the code and degree of specialization required. But if anecdotal evidence may be permitted, I would hazard that this is more likely than one would intuitively expect. In the Subversion project, we (the founders) believed we faced a deep and complex set of problems, which we had been thinking about hard for several months, and we frankly doubted that anyone on the newly created mailing list was likely to make a real contribution to the discussion. So we took the lazy route and started batting some technical ideas back and forth in private emails, until an observer of the project[1] caught wind of what was happening and asked for the discussion to be moved to the public list. Rolling our eyes a bit, we did—and were stunned by the number of insightful comments and suggestions that quickly resulted. In many cases people offered ideas that had never even occurred to us. It turned out there were some *very* smart people on that list; they'd just been waiting for the right bait. It's true that the ensuing discussions took longer than they would have if we had kept the conversation private, but they were so much more productive that it was well worth the extra time.

Without descending into hand-waving generalizations like "the group is always smarter than the individual" (we've all met enough groups to know better), it must be acknowledged that there are certain activities at which groups excel. Massive peer review is one of them; generating large numbers of ideas quickly is another. The quality of the ideas depends on the quality of the thinking that went into them, of course, but you won't know what kinds of thinkers are out there until you stimulate them with a challenging problem.

Naturally, there are some discussions that must be had privately; throughout this book we'll see examples of those. But the guiding principle should always be: *If there's no reason for it to be private, it should be public.*

Making this happen requires action. It's not enough merely to ensure that all your own posts go to the public list. You also have to nudge other people's unnecessarily private conversations to the list too. If someone tries to start a private discussion, and there's no reason for it to be private, then it is incumbent on you to open the appropriate meta-discussion immediately. Don't even comment on the original topic until you've either successfully steered the conversation to a public place, or ascertained that privacy really was needed. If you do this consistently, people will catch on pretty quickly and start to use the public forums by default.

Nip Rudeness in the Bud

From the very start of your project's public existence, you should maintain a zero-tolerance policy toward rude or insulting behavior in its forums. Zero-tolerance does not mean technical enforcement per se. You don't have to remove people from the mailing list when they flame another subscriber, or take away their commit access because they made derogatory comments. (In theory, you might eventually have to resort to such actions, but only after all other avenues have failed—which, by definition, isn't the case at the start of the project.) Zero-tolerance simply means never letting bad behavior slide by unnoticed. For example, when someone posts a technical comment mixed together with an *ad hominem* attack on some other developer in the project, it is imperative that your response address the *ad hominem* attack *first*, as a separate issue unto itself, and only afterward move on to the technical content.

It is unfortunately very easy, and all too typical, for constructive discussions to lapse into destructive flame wars. People will say things in email that they would never say face-to-face. The topics of discussion only amplify this effect: in technical issues, people often feel there is a single right answer to most questions, and that disagreement with that answer can only be explained by ignorance or stupidity. It's a short distance from calling someone's technical proposal stupid to calling the person themselves stupid. In fact, it's often hard to tell where technical debate leaves off and character attack begins, which is one reason why drastic responses or punishments are not a good idea. Instead, when you think you see it happening, make a post that stresses the importance of keeping the discussion friendly, without accusing

[1] We haven't gotten to the section on crediting yet, but just to practice what I'll later preach: the observer's name was Brian Behlendorf, and it was he who pointed out the general importance of keeping all discussions public unless there was a specific need for privacy.

anyone of being deliberately poisonous. Such "Nice Police" posts do have an unfortunate tendency to sound like a kindergarten teacher lecturing a class on good behavior:

> *First, let's please cut down on the (potentially) ad hominem comments; for example, calling J's design for the security layer "naive and ignorant of the basic principles of computer security." That may be true or it may not, but in either case it's no way to have the discussion. J made his proposal in good faith. If it has deficiencies, point them out, and we'll fix them or get a new design. I'm sure M meant no personal insult to J, but the phrasing was unfortunate, and we try to keep things constructive around here.*
>
> *Now, on to the proposal. I think M was right in saying that...*

As stilted as such responses sound, they have a noticeable effect. If you consistently call out bad behavior, but don't demand an apology or acknowledgment from the offending party, then you leave people free to cool down and show their better side by behaving more decorously next time—and they will. One of the secrets of doing this successfully is to never make the meta-discussion the main topic. It should always be an aside, a brief preface to the main portion of your response. Point out in passing that "we don't do things that way around here," but then move on to the real content, so that you're giving people something on-topic to respond to. If someone protests that they didn't deserve your rebuke, simply refuse to be drawn into an argument about it. Either don't respond (if you think they're just letting off steam and don't require a response), or say you're sorry if you overreacted and that it's hard to detect nuance in email, then get back to the main topic. Never, ever insist on an acknowledgment, whether public or private, from someone that they behaved inappropriately. If they choose of their own volition to post an apology, that's great, but demanding that they do so will only cause resentment.

The overall goal is to make good etiquette be seen as one of the "in-group" behaviors. This helps the project, because developers can be driven away (even from projects they like and want to support) by flame wars. You may not even know that they were driven away; someone might lurk on the mailing list, see that it takes a thick skin to participate in the project, and decide against getting involved at all. Keeping forums friendly is a long-term survival strategy, and it's easier to do when the project is still small. Once it's part of the culture, you won't have to be the only person promoting it. It will be maintained by everyone.

Practice Conspicuous Code Review

One of the best ways to foster a productive development community is to get people looking at each others' code. Some technical infrastructure is required to do this effectively—in particular, commit emails must be turned on; see the section called “Commit emails” for more details. The effect of commit emails is that every time someone commits a change to the source code, an email goes out showing the log message and diffs for the change (see *diff*, in the section called “Version Control Vocabulary”). *Code review* is the practice of reviewing commit emails as they come in, looking for bugs and possible improvements.[2]

Code review serves several purposes simultaneously. It's the most obvious example of peer review in the open source world, and directly helps to maintain software quality. Every bug that ships in a piece of software got there by being committed and not detected; therefore, the more eyes watch commits, the fewer bugs will ship. But code review also serves an indirect purpose: it confirms to people that what they do matters, because one obviously wouldn't take time to review a commit unless one cared about its effect. People do their best work when they know that others will take the time to evaluate it.

Reviews should be public. Even on occasions when I have been sitting in the same physical room with developers, and one of us has made a commit, we take care not to do the review verbally in the room, but to send it to the development mailing list instead. Everyone benefits from seeing the review happen.

[2]This is how code review is usually done in open source projects, at any rate. In more centralized projects, "code review" can also mean multiple people sitting down together and going over printouts of source code, looking for specific problems and patterns.

People follow the commentary and sometimes find flaws in it, and even when they don't, it still reminds them that review is an expected, regular activity, like washing the dishes or mowing the lawn.

In the Subversion project, we did not at first make a regular practice of code review. There was no guarantee that every commit would be reviewed, though one might sometimes look over a change if one was particularly interested in that area of the code. Bugs slipped in that really could and should have been caught. A developer named Greg Stein, who knew the value of code review from past work, decided that he was going to set an example by reviewing every line of *every single commit* that went into the code repository. Each commit anyone made was soon followed by an email to the developer's list from Greg, dissecting the commit, analyzing possible problems, and occasionally praising a clever bit of code. Right away, he was catching bugs and non-optimal coding practices that would otherwise have slipped by without ever being noticed. Pointedly, he never complained about being the only person reviewing every commit, even though it took a fair amount of his time, but he did sing the praises of code review whenever he had the chance. Pretty soon, other people, myself included, started reviewing commits regularly too. What was our motivation? It wasn't that Greg had consciously shamed us into it. But he had proven that reviewing code was a valuable way to spend time, and that one could contribute as much to the project by reviewing others' changes as by writing new code. Once he demonstrated that, it became expected behavior, to the point where any commit that didn't get some reaction would cause the committer to worry, and even ask on the list whether anyone had had a chance to review it yet. Later, Greg got a job that didn't leave him as much time for Subversion, and had to stop doing regular reviews. But by then, the habit was so ingrained for the rest of us as to seem that it had been going on since time immemorial.

Start doing reviews from very first commit. The sorts of problems that are easiest to catch by reviewing diffs are security vulnerabilities, memory leaks, insufficient comments or API documentation, off-by-one errors, caller/callee discipline mismatches, and other problems that require a minimum of surrounding context to spot. However, even larger-scale issues such as failure to abstract repeated patterns to a single location become spottable after one has been doing reviews regularly, because the memory of past diffs informs the review of present diffs.

Don't worry that you might not find anything to comment on, or that you don't know enough about every area of the code. There will usually be something to say about almost every commit; even where you don't find anything to question, you may find something to praise. The important thing is to make it clear to every committer that what they do is seen and understood. Of course, code review does not absolve programmers of the responsibility to review and test their changes before committing; no one should depend on code review to catch things he ought to have caught on his own.

When Opening a Formerly Closed Project, be Sensitive to the Magnitude of the Change

If you're opening up an existing project, one that already has active developers accustomed to working in a closed-source environment, make sure everyone understands that a big change is coming—and make sure that you understand how it's going to feel from their point of view.

Try to imagine how the situation looks to them: formerly, all code and design decisions were made with a group of other programmers who knew the software more or less equally well, who all received the same pressures from the same management, and who all know each others' strengths and weaknesses. Now you're asking them to expose their code to the scrutiny of random strangers, who will form judgements based only on the code, with no awareness of what business pressures may have forced certain decisions. These strangers will ask lots of questions, questions that jolt the existing developers into realizing that the documentation they slaved so hard over is *still* inadequate (this is inevitable). To top it all off, the newcomers are unknown, faceless entities. If one of your developers already feels insecure about his skills, imagine how that will be exacerbated when newcomers point out flaws in code he wrote, and worse, do so in front of his colleagues. Unless you have a team of perfect coders, this is unavoidable—in fact, it will probably happen to all of them at first. This is not because they're bad programmers; it's just that any program above a certain size has bugs, and peer review will spot some of those bugs (see the

section called “Practice Conspicuous Code Review” earlier in this chapter). At the same time, the newcomers themselves won't be subject to much peer review at first, since they can't contribute code until they're more familiar with the project. To your developers, it may feel like all the criticism is incoming, never outgoing. Thus, there is the danger of a siege mentality taking hold among the old hands.

The best way to prevent this is to warn everyone about what's coming, explain it, tell them that the initial discomfort is perfectly normal, and reassure them that it's going to get better. Some of these warnings should take place privately, before the project is opened. But you may also find it helpful to remind people on the public lists that this is a new way of development for the project, and that it will take some time to adjust. The very best thing you can do is lead by example. If you don't see your developers answering enough newbie questions, then just telling them to answer more isn't going to help. They may not have a good sense of what warrants a response and what doesn't yet, or it could be that they don't have a feel for how to prioritize coding work against the new burden of external communications. The way to get them to participate is to participate yourself. Be on the public mailing lists, and make sure to answer some questions there. When you don't have the expertise to field a question, then visibly hand it off to a developer who does—and watch to make sure he follows up with an answer, or at least a response. It will naturally be tempting for the longtime developers to lapse into private discussions, since that's what they're used to. Make sure you're subscribed to the internal mailing lists on which this might happen, so you can ask that such discussions be moved to the public lists right away.

There are other, longer-term concerns with opening up formerly closed projects. Chapter 4, *Social and Political Infrastructure* explores techniques for mixing paid and unpaid developers successfully, and Chapter 9, *Licenses, Copyrights, and Patents* discusses the necessity of legal diligence when opening up a private code base that may contain software written or "owned" by other parties.

Announcing

Once the project is presentable—not perfect, just presentable—you're ready to announce it to the world. This is actually a very simple process: go to http://freshmeat.net/, click on Submit in the top navigation bar, and fill out a form announcing your new project. Freshmeat is the place everyone watches for new project announcements. You only have to catch a few eyes there for news of your project to spread by word of mouth.

If you know of mailing lists or newsgroups where an announcement of your project would be on-topic and of interest, then post there, but be careful to make exactly *one* post per forum, and to direct people to your project's own forums for follow-up discussion (by setting the `Reply-to` header). The posts should be short and get right to the point:

```
To: discuss@lists.example.org
Subject: [ANN] Scanley full-text indexer project
Reply-to: dev@scanley.org

This is a one-time post to announce the creation of the Scanley
project, an open source full-text indexer and search engine with a
rich API, for use by programmers in providing search services for
large collections of text files.  Scanley is now running code, is
under active development, and is looking for both developers and
testers.

Home page: http://www.scanley.org/

Features:
   - Searches plain text, HTML, and XML
   - Word or phrase searching
   - (planned) Fuzzy matching
   - (planned) Incremental updating of indexes
   - (planned) Indexing of remote web sites

Requirements:
```

```
   - Python 2.2 or higher
   - Enough disk space to hold the indexes (approximately 2x
     original data size)

For more information, please come to scanley.org.

Thank you,
-J. Random
```

(See the section called “Publicity” in Chapter 6, *Communications* for advice on announcing further releases and other project events.)

There is an ongoing debate in the free software world about whether it is necessary to begin with running code, or whether a project can benefit from being opened even during the design/discussion stage. I used to think starting with running code was the most important factor, that it was what separated successful projects from toys, and that serious developers would only be attracted to software that did something concrete already.

This turned out not to be the case. In the Subversion project, we started with a design document, a core of interested and well-connected developers, a lot of fanfare, and *no* running code at all. To my complete surprise, the project acquired active participants right from the beginning, and by the time we did have something running, there were quite a few volunteer developers already deeply involved. Subversion is not the only example; the Mozilla project was also launched without running code, and is now a successful and popular web browser.

In the face of such evidence, I have to back away from the assertion that running code is absolutely necessary for launching a project. Running code is still the best foundation for success, and a good rule of thumb would be to wait until you have it before announcing your project. However, there may be circumstances where announcing earlier makes sense. I do think that at least a well-developed design document, or else some sort of code framework, is necessary—of course it may be revised based on public feedback, but there has to be something concrete, something more tangible than just good intentions, for people to sink their teeth into.

Whenever you announce, don't expect a horde of volunteers to join the project immediately afterward. Usually, the result of announcing is that you get a few casual inquiries, a few more people join your mailing lists, and aside from that, everything continues pretty much as before. But over time, you will notice a gradual increase in participation from both new code contributors and users. Announcement is merely the planting of a seed. It can take a long time for the news to spread. If the project consistently rewards those who get involved, the news *will* spread, though, because people want to share when they've found something good. If all goes well, the dynamics of exponential communications networks will slowly transform the project into a complex community, where you don't necessarily know everyone's name and can no longer follow every single conversation. The next chapters are about working in that environment.